

Contenido

Volatile	1
Volatile y Synchroized	4
AtomicInteger.....	5
Wait.....	7
Notify NotifyAll	9
BlockingQueue ArrayBlockingQueue.....	10
Callable Future Futuretask	13
ThreadPool.....	15
Executors.....	17
ScheduledExecutorService	19

Volatile

Uso de Volatile

- Es una **palabra clave** que se aplica a variables compartidas entre múltiples hilos.
- Indica que la variable **no debe ser cacheada por los hilos** en registros o memoria local, sino que siempre debe leerse y escribirse directamente desde la memoria principal.
- Garantiza **visibilidad**: cuando un hilo modifica la variable, otros hilos ven inmediatamente el cambio.

Volatile asegura

- **Visibilidad de cambios:** todos los hilos leen el valor actualizado.
- **No sincronización completa:** no evita condiciones de carrera si hay operaciones compuestas (ejemplo: contador++).
- **Uso típico:** banderas de control, estados de ejecución, variables que cambian de forma simple (true/false, referencias).

No hace

- No asegura **atomicidad**. Si necesitas operaciones seguras (como incrementar un contador), debes usar `synchronized` o clases de concurrencia como `AtomicInteger`.
- No reemplaza mecanismos de sincronización más complejos.

Ejemplo conceptual

Imagina que tienes un hilo que corre en bucle hasta que una variable cambie a `false`.

- Sin `volatile`, ese hilo podría seguir corriendo porque nunca “ve” el cambio (usa una copia vieja).
- Con `volatile`, el hilo detecta el cambio inmediatamente y se detiene.

En resumen: Usa `volatile` cuando quieras que **todos los hilos tengan siempre la misma visión de una variable simple**, pero no cuando necesites operaciones compuestas seguras.

```
class HiloTrabajador extends Thread {
    private volatile boolean ejecutar;
    private int id;
    private int contador = 0;

    public HiloTrabajador(int id_) {
        ejecutar = true;
        id = id_;
    }

    @Override
    public void run() {
        System.out.println("Hilo trabajador iniciado: " + id );
        int contador = 0;

        while (ejecutar) {
            contador++;
            //if (contador % 10000000000 == 0) {
            //    System.out.println("Hilo " + id + " lleva " +
            contador + " iteraciones");
        }
    }
}
```

```

        //}
    }

    System.out.println("Hilo trabajador " + id + " terminado.
Contador: " + contador);
}

    public void detener() {
        System.out.println("Deteniendo hilo trabajador " + id + "
...");
        this.ejecutar = false;
    }
}

public class EjemVolatile {

    public static void main(String[] args) throws InterruptedException
    {
        int H = 3;
        HiloTrabajador[] trabajadores = new HiloTrabajador[H];

        // Crear el hilo trabajadores
        // Iniciar el hilos
        for (int i = 0; i < H; i++) {
            trabajadores[i] = new HiloTrabajador(i + 1);
            trabajadores[i].start();
        }

        // Dormimos 3 segundos
        Thread.sleep(3000);

        // Detener hilos mediante el metodo que cambia volatile
        for (int i = 0; i < H; i++) {
            trabajadores[i].detener();
        }

        // Esperamos a que los hilos terminen
        for (int i = 0; i < H; i++) {
            trabajadores[i].join();
        }

        System.out.println("Programa finalizado correctamente");
    }
}

```

Volatile y Synchronized

- volatile asegura **visibilidad inmediata** de los cambios en una variable compartida entre hilos.
- synchronized asegura **atomicidad y exclusión mutua**, evitando que varios hilos ejecuten al mismo tiempo una operación crítica.
- Juntos, se pueden usar cuando necesitas que una variable sea visible para todos los hilos y además proteger operaciones compuestas sobre ella.

```
class Contador {
    // Volatile asegura visibilidad entre hilos
    private volatile int valor = 0;

    // synchronized asegura atomicidad en la operación
    public synchronized void incrementar() {
        valor++; // operación segura
    }

    public synchronized int getValor() {
        return valor;
    }
}

public class EjemVolatileYSynchronized {
    public static void main(String[] args) throws InterruptedException
    {
        final Contador contador = new Contador();

        // Crear varios hilos usando clases anónimas (sin "->")
        Thread hilo1 = new Thread(new Runnable() {
            @Override
            public void run() {
                for (int i = 0; i < 1000; i++) {
                    contador.incrementar();
                }
            }
        });

        Thread hilo2 = new Thread(new Runnable() {
            @Override
            public void run() {
                for (int i = 0; i < 1000; i++) {
                    contador.incrementar();
                }
            }
        });
    }
}
```

```

Thread hilo3 = new Thread(new Runnable() {
    @Override
    public void run() {
        for (int i = 0; i < 1000; i++) {
            contador.incrementar();
        }
    }
});

// Iniciar los hilos
hilo1.start();
hilo2.start();
hilo3.start();

// Esperar a que terminen
hilo1.join();
hilo2.join();
hilo3.join();

System.out.println("Valor esperado: 3000");
System.out.println("Valor real: " + contador.getValor());
}
}

```

AtomicInteger

- AtomicInteger es una clase de la librería `java.util.concurrent.atomic`.
- Permite realizar operaciones sobre enteros de manera **atómica** (segura en entornos concurrentes).
- A diferencia de usar `volatile` o variables normales, AtomicInteger asegura que operaciones como `incrementAndGet()` o `addAndGet()` no se solapen entre hilos.
- Se usa mucho en aplicaciones multihilo donde varios hilos deben modificar un contador o estado compartido.
-

Métodos de AtomicInteger	Descripción
<code>get()</code>	Devuelve el valor actual.
<code>set(int newValue)</code>	Establece un nuevo valor.
<code>lazySet(int newValue)</code>	Similar a <code>set</code> , pero puede retrasar la escritura en memoria para mayor eficiencia.
<code>getAndSet(int newValue)</code>	Devuelve el valor actual y luego lo reemplaza por el nuevo.
<code>incrementAndGet()</code>	Incrementa en 1 y devuelve el nuevo valor.

getAndIncrement()	Devuelve el valor actual y luego lo incrementa en 1.
decrementAndGet()	Decrementa en 1 y devuelve el nuevo valor.
getAndDecrement()	Devuelve el valor actual y luego lo decrementa en 1.
addAndGet(int delta)	Suma el valor indicado y devuelve el nuevo valor.
getAndAdd(int delta)	Devuelve el valor actual y luego suma el valor indicado.
compareAndSet(expect, update)	Cambia el valor solo si coincide con el esperado. Devuelve true si lo logra.
weakCompareAndSet(expect, update)	Similar a compareAndSet, pero puede fallar espuriamente (más rápido).

- Los métodos get y set son básicos para leer y escribir.
- Los métodos incrementAndGet, decrementAndGet, addAndGet permiten operaciones aritméticas seguras.
- Los métodos compareAndSet y weakCompareAndSet son útiles para sincronización sin bloqueos.

```
import java.util.concurrent.atomic.AtomicInteger;

class ContadorAtomic {
    private AtomicInteger valor = new AtomicInteger(0);

    public void incrementar() {
        valor.incrementAndGet(); // operación atómica
    }

    public int getValor() {
        return valor.get();
    }
}

public class EjemAtomicInteger {
    public static void main(String[] args) throws InterruptedException
    {
        final ContadorAtomic contador = new ContadorAtomic();

        // Crear dos hilos usando clases anónimas (sin "->")
        Thread hilo1 = new Thread(new Runnable() {
            @Override
            public void run() {
                for (int i = 0; i < 1000; i++) {
                    contador.incrementar();
                }
            }
        });

        Thread hilo2 = new Thread(new Runnable() {
```

```

        @Override
        public void run() {
            for (int i = 0; i < 1000; i++) {
                contador.incrementar();
            }
        }
    });

    // Iniciar los hilos
    hilo1.start();
    hilo2.start();

    // Esperar a que terminen
    hilo1.join();
    hilo2.join();

    System.out.println("Valor esperado: 2000");
    System.out.println("Valor real: " + contador.getValor());
}
}

```

Wait

El método `wait()` en Java se usa junto con `synchronized` para que un hilo se quede esperando hasta que otro hilo lo despierte con `notify()` o `notifyAll()`. Es una forma de coordinación entre varios hilos que comparten un mismo recurso.

- `wait()`: el hilo libera el bloqueo del objeto y queda en espera.
- `notify()`: despierta un hilo que estaba esperando en ese objeto.
- `notifyAll()`: despierta a todos los hilos que estaban esperando.
- Solo se puede llamar a `wait()` y `notify()` dentro de un bloque `synchronized`, porque dependen del monitor del objeto.

```

class RecursoCompartido {
    private boolean disponible = false;

    public synchronized void producir() {
        System.out.println("Productor: creando recurso...");
        disponible = true;
        notify(); // Avisar a un consumidor que ya hay recurso
    }

    public synchronized void consumir() {

```

```

        while (!disponible) {
            try {
                System.out.println("Consumidor: esperando
recurso...");
                wait(); // Esperar hasta que el productor avise
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        System.out.println("Consumidor: recurso consumido!");
        disponible = false;
    }
}

public class EjemWait {
    public static void main(String[] args) {
        RecursoCompartido recurso = new RecursoCompartido();

        // Hilo consumidor
        Thread consumidor = new Thread(new Runnable() {
            @Override
            public void run() {
                recurso.consumir();
            }
        });

        // Hilo productor
        Thread productor = new Thread(new Runnable() {
            @Override
            public void run() {
                try {
                    Thread.sleep(2000); // Simular tiempo de
producción
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
                recurso.producir();
            }
        });

        consumidor.start();
        productor.start();
    }
}

```

- El **consumidor** intenta consumir, pero como no hay recurso (disponible = false), entra en wait().
- El **productor** después de 2 segundos produce el recurso y llama a notify().
- El consumidor se despierta, consume el recurso y termina.

Notify NotifyAll

La diferencia entre notify() y notifyAll() en Java está en **cuántos hilos en espera son despertados** cuando se invoca el método sobre un objeto sincronizado:

notify()

- Despierta **solo un hilo** que esté esperando en el monitor del objeto.
- No se puede controlar cuál hilo será despertado (la elección depende de la JVM).
- Útil cuando sabes que basta con que un único consumidor continúe.

notifyAll()

- Despierta **todos los hilos** que estén esperando en el monitor del objeto.
- Luego, cada hilo competirá por adquirir el bloqueo y continuar.
- Útil cuando hay varios consumidores y quieres darles la oportunidad de reaccionar.

```
class Recurso {

    private boolean disponible = false;

    public synchronized void producir() {
        disponible = true;
        System.out.println("Productor: recurso disponible");
        notify();    // Solo despierta a un hilo
        //notifyAll();    // Despierta a todos los hilos
    }

    public synchronized void consumir(String nombre) {
        while (!disponible) {
            try {
                System.out.println(nombre + " esperando recurso...");
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        System.out.println(nombre + " consumió el recurso");
        disponible = false;
    }
}

public class EjmNotifyNotifyAll {
```

```

public static void main(String[] args) {
    Recurso recurso = new Recurso();

    // Varios consumidores
    for (int i = 1; i <= 3; i++) {
        final String nombre = "Consumidor-" + i;
        new Thread(new Runnable() {
            @Override
            public void run() {
                recurso.consumir(nombre);
            }
        }).start();
    }

    // Productor
    new Thread(new Runnable() {
        @Override
        public void run() {
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
            recurso.producir();
        }
    }).start();
}

```

- Con `notify()`: solo uno de los consumidores se despierta y consume el recurso; los demás siguen esperando.
- Con `notifyAll()`: todos los consumidores se despiertan, pero solo uno logrará consumir (porque el recurso se marca como no disponible después). Los otros volverán a esperar.

En resumen:

- Usa `notify()` cuando solo necesitas despertar a un consumidor.
- Usa `notifyAll()` cuando hay varios hilos esperando y quieres darles la oportunidad de competir por el recurso.

BlockingQueue ArrayBlockingQueue

BlockingQueue

- Es una **interfaz** que define el comportamiento de una cola bloqueante.
- No se puede instanciar directamente, pero permite programar contra la abstracción.
- Puedes cambiar fácilmente la implementación (por ejemplo, ArrayBlockingQueue, LinkedBlockingQueue, etc.) sin modificar el resto del código.

ArrayBlockingQueue

- Es una **clase concreta** que implementa BlockingQueue.
- Internamente usa un **array circular fijo**.
- La capacidad se define al crearla y no puede cambiar.
- Es eficiente, pero siempre acotada.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.ArrayBlockingQueue;

public class EjmArrayBlockingQueue {
    public static void main(String[] args) throws InterruptedException
    {
        // Cola declarada como interfaz BlockingQueue (capacidad 2)
        BlockingQueue<Integer> colaOperaciones = new
        ArrayBlockingQueue<>(2);

        // Cola declarada directamente como ArrayBlockingQueue
        (capacidad 3)
        ArrayBlockingQueue<Integer> colaNumeros = new
        ArrayBlockingQueue<>(3);

        // --- Productor de operaciones aritméticas ---
        for (int i = 1; i <= 3; i++) {
            System.out.println("Productor de operaciones: poniendo " +
i);
            colaOperaciones.put(i); // la tercera se bloqueará hasta
que alguien consuma
        }

        // --- Consumidor de operaciones ---
        Thread consumidorOperaciones = new Thread(new Runnable() {
            public void run() {
                try {
                    while (true) {
                        int valor = colaOperaciones.take();
                        System.out.println("Consumidor de operaciones:
sacó " + valor);
                    }
                }
            }
        });
        consumidorOperaciones.start();
    }
}
```

```

        int resultado = 0;
        // bucle con operación aritmética: suma de
cuadrados
        for (int j = 1; j <= valor; j++) {
            resultado += j * j;
        }
        System.out.println("Suma de cuadrados hasta "
+ valor + " = " + resultado);
    }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
});
consumidorOperaciones.start();

// --- Productor de números ---
for (int i = 1; i <= 4; i++) {
    System.out.println("Productor de números: poniendo " + i);
    colaNumeros.put(i); // el cuarto se bloqueará hasta que
alguien consuma
}

// --- Consumidor de números ---
Thread consumidorNumeros = new Thread(new Runnable() {
    public void run() {
        try {
            while (true) {
                int num = colaNumeros.take();
                System.out.println("Consumidor de números:
sacó " + num);

                int factorial = 1;
                // bucle con operación aritmética: calcular
factorial
                for (int j = 1; j <= num; j++) {
                    factorial *= j;
                }
                System.out.println("Factorial de " + num + " =
" + factorial);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
});
consumidorNumeros.start();
}
}

```

BlockingQueue (**colaOperaciones**):

- Capacidad = 2.

- Se insertan 3 valores. El tercero se bloquea hasta que el consumidor saque uno.
- El consumidor hace un bucle para calcular la **suma de cuadrados** hasta ese número.

ArrayBlockingQueue (**colaNumeros**):

- Capacidad = 3.
- Se insertan 4 valores. El cuarto se bloquea hasta que el consumidor saque uno.
- El consumidor hace un bucle para calcular el **factorial** del número extraído.

BlockingQueue es la abstracción, útil para programar de forma flexible.

ArrayBlockingQueue es una implementación concreta con capacidad fija.

Ambas bloquean al productor si se excede la capacidad y al consumidor si la cola está vacía.

En este ejemplo, cada cola recibe **tareas de bucles con operaciones aritméticas distintas**: suma de cuadrados y factorial.

Callable Future Futuretask

Callable<V>

- Es una interfaz similar a Runnable, pero su método call() devuelve un resultado y puede lanzar excepciones.
- Se usa cuando necesitas que una tarea retorne un valor.

Future<V>

- Es una interfaz que representa el resultado de una tarea asíncrona.
- Permite consultar si la tarea terminó (isDone()), cancelarla (cancel()), y obtener el resultado (get()).
- Se obtiene normalmente al enviar un Callable a un ExecutorService.

FutureTask<V>

- Es una clase concreta que implementa Runnable y Future.
- Puede envolver un Callable o un Runnable.
- Se puede ejecutar directamente en un hilo (new Thread(new FutureTask(...)).start()) o enviarse a un ExecutorService.
- Es útil cuando quieres manejar tanto la ejecución como el resultado en un solo objeto

```
import java.util.concurrent.*;

public class EjemCallableFutureTask {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        // --- 1. Callable ---
        Callable<Integer> sumaCallable = new Callable<Integer>() {
            public Integer call() {
                int suma = 0;
                for (int i = 1; i <= 5; i++) {
                    suma += i; // suma de 1 a 5
                }
                return suma;
            }
        };

        // --- 2. Future ---
        Future<Integer> resultadoFuture =
        executor.submit(sumaCallable);
        System.out.println("Resultado obtenido con Future (suma): " +
        resultadoFuture.get());

        // --- 3. FutureTask ---
        FutureTask<Integer> factorialTask = new FutureTask<>(new
        Callable<Integer>() {
            public Integer call() {
                int factorial = 1;
                for (int i = 1; i <= 5; i++) {
                    factorial *= i; // factorial de 5
                }
                return factorial;
            }
        });

        Thread hilo = new Thread(factorialTask);
        hilo.start();
    }
}
```

```
        System.out.println("Resultado obtenido con FutureTask  
(factorial): " + factorialTask.get());  
  
        executor.shutdown();  
    }  
}
```

1. **Callable:** define la tarea de sumar números del 1 al 5.
2. **Future:** recibe el resultado de esa tarea cuando se ejecuta en un `ExecutorService`. Con `get()` recuperamos el valor.
3. **FutureTask:** envuelve un `Callable` que calcula el factorial de 5. Se ejecuta directamente en un hilo y luego se obtiene el resultado con `get()`.

Uso diferentes:

Callable: define la lógica de la tarea que devuelve un valor.

Future: representa el resultado pendiente de esa tarea, útil para esperar o consultar su estado.

FutureTask: combina ejecución y resultado en un solo objeto, y puede usarse tanto en un `ExecutorService` como en un hilo directo.

la utilidad diferenciada de cada uno:

- `Callable` para definir tareas con retorno,
- `Future` para manejar resultados asíncronos,
- `FutureTask` para tener control total sobre ejecución y resultado.

ThreadPool

Un **ThreadPool** es un conjunto de hilos que ejecutan tareas. En algunos casos, puede ser útil tener **más de un pool**:

- **Separación de responsabilidades**: por ejemplo, un pool para tareas de cálculo intensivo y otro para tareas de entrada/salida.
- **Control de recursos**: evitar que tareas pesadas bloqueen otras más ligeras.
- **Priorización**: asignar diferentes tamaños de pool según la importancia de las tareas.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class EjmThreadPools {
    public static void main(String[] args) throws InterruptedException
    {
        // Primer ThreadPool: más hilos, tareas rápidas
        ExecutorService pool1 = Executors.newFixedThreadPool(4);

        // Variable compartida para acumular resultados
        final int[] suma = {0};

        // Lanzar varias tareas de cálculo sencillo
        for (int i = 1; i <= 6; i++) {
            final int num = i;
            pool1.submit(new Runnable() {
                @Override
                public void run() {
                    System.out.println("Pool1 procesando " + num +
                                         " en " +
Thread.currentThread().getName());
                    try {
                        Thread.sleep(500); // Simular trabajo ligero
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                    synchronized (suma) {
                        suma[0] += num * 2; // Acumular resultado
                    }
                }
            });
        }

        // Esperar a que terminen todas las tareas del primer pool
        pool1.shutdown();
        pool1.awaitTermination(10, TimeUnit.SECONDS);
    }
}
```

```

        System.out.println("Resultado combinado del Pool1: " +
suma[0]);

        // Segundo ThreadPool: menos hilos, tareas más pesadas
        ExecutorService pool2 = Executors.newFixedThreadPool(2);

        // Usar el resultado del primer pool como entrada
        for (int i = 1; i <= 3; i++) {
            final int tareaId = i;
            final int entrada = suma[0];
            pool2.submit(new Runnable() {
                @Override
                public void run() {
                    System.out.println("Pool2 procesando tarea " +
tareaId +
                                " con entrada " + entrada +
                                " en " +
Thread.currentThread().getName());
                    // Trabajo aritmético costoso
                    long resultado = 0;
                    for (int j = 1; j <= 5000000; j++) {
                        resultado += (entrada * j) % 97;
                    }
                    System.out.println("Tarea " + tareaId +
                                " completada con resultado " +
resultado);
                }
            });
        }

        pool2.shutdown();
    }
}

```

- Creamos **dos pools**: uno con 2 hilos y otro con 3.
- Las tareas de cálculo se ejecutan en el primer pool, más limitado.
- Las tareas de IO se ejecutan en el segundo pool, con más hilos disponibles.
- Ambos pools trabajan en paralelo, pero **no se bloquean entre sí**.

Executors

En Java, la clase Executors es una fábrica que nos permite crear distintos tipos de ExecutorService, es decir, gestores de hilos que ejecutan tareas en segundo plano.

En lugar de crear hilos manualmente con `new Thread(...)`, usamos `Executors` para simplificar y controlar mejor la concurrencia.

- `Executors.newFixedThreadPool(n)`: crea un pool con `n` hilos fijos.
- `Executors.newCachedThreadPool()`: crea un pool dinámico que ajusta el número de hilos según la carga.
- `Executors.newSingleThreadExecutor()`: crea un pool con un solo hilo, útil para ejecutar tareas en orden.
- `Executors.newScheduledThreadPool(n)`: permite programar tareas periódicas o con retraso.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class EjemExecutors {
    public static void main(String[] args) {
        // Primer Executor con 2 hilos
        ExecutorService executor1 =
            Executors.newFixedThreadPool(2);

        // Variable compartida para guardar resultado
        final int[] resultado = {0};

        // Ejecutar tarea inicial
        executor1.execute(new Runnable() {
            @Override
            public void run() {
                System.out.println("Executor1: realizando cálculo
inicial...");

                int suma = 0;
                for (int i = 1; i <= 5; i++) {
                    suma += i * 2; // operación sencilla
                }
                resultado[0] = suma;
                System.out.println("Executor1: resultado = " + suma);
            }
        });

        // Cerrar el primer executor
        executor1.shutdown();
        while (!executor1.isTerminated()) {
            // Esperar a que termine
        }

        // Segundo Executor con 1 hilo (más costoso)
        ExecutorService executor2 =
            Executors.newFixedThreadPool(1);

        // Usar el resultado del primero
```

```

        executor2.execute(new Runnable() {
            @Override
            public void run() {
                System.out.println("Executor2: usando resultado " +
resultado[0]);
                long calculoPesado = 0;
                for (int j = 1; j <= 3000000; j++) {
                    calculoPesado += (resultado[0] * j) % 97;
                }
                System.out.println("Executor2: cálculo pesado
terminado, resultado = " + calculoPesado);
            }
        });

        executor2.shutdown();
    }
}

```

- Executor1 (2 hilos): hace un cálculo inicial sencillo (sumar números).
- Se espera a que termine y se guarda el resultado en una variable compartida.
- Executor2 (1 hilo): recibe ese resultado y realiza un cálculo más pesado (millones de operaciones).
- Cada executor trabaja de manera independiente, pero se encadenan en secuencia.

ScheduledExecutorService

El `ScheduledExecutorService` en Java es una extensión de `ExecutorService` que permite **programar tareas** para que se ejecuten después de un retraso o de forma periódica. Es muy útil para temporizadores, tareas repetitivas o procesos que deben ejecutarse cada cierto intervalo.

- `schedule(Runnable, delay, TimeUnit)`: ejecuta una tarea después de un retraso.
- `scheduleAtFixedRate(Runnable, initialDelay, period, TimeUnit)`: ejecuta una tarea periódicamente con un intervalo fijo.
- `scheduleWithFixedDelay(Runnable, initialDelay, delay, TimeUnit)`: ejecuta una tarea periódicamente, pero el tiempo de espera se cuenta desde que terminó la tarea anterior.

```

import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class EjemScheduledExecutor {
    public static void main(String[] args) {
        // Crear un ScheduledExecutor con 3 hilos
        ScheduledExecutorService scheduler =
        Executors.newScheduledThreadPool(3);

        // Tarea 1: ejecutar una vez después de 2 segundos
        scheduler.schedule(new Runnable() {
            @Override
            public void run() {
                System.out.println("Tarea 1 ejecutada en " +
                Thread.currentThread().getName());
            }
        }, 2, TimeUnit.SECONDS);

        // Tarea 2: ejecutar cada 3 segundos, empezando después de 1
        segundo
        scheduler.scheduleAtFixedRate(new Runnable() {
            @Override
            public void run() {
                System.out.println("Tarea 2 ejecutada en " +
                Thread.currentThread().getName());
            }
        }, 1, 3, TimeUnit.SECONDS);

        // Tarea 3: ejecutar cada 4 segundos, pero contando desde que
        termina la tarea
        scheduler.scheduleWithFixedDelay(new Runnable() {
            @Override
            public void run() {
                System.out.println("Tarea 3 ejecutada en " +
                Thread.currentThread().getName());
                try {
                    Thread.sleep(2000); // Simular trabajo costoso
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }, 1, 4, TimeUnit.SECONDS);

        // El scheduler seguirá ejecutando tareas en segundo plano
        // Para detenerlo después de cierto tiempo:
        scheduler.schedule(new Runnable() {
            @Override
            public void run() {
                System.out.println("Apagando scheduler...");
                scheduler.shutdown();
            }
        }, 20, TimeUnit.SECONDS);
    }
}

```

```
}  
}
```

- Se crea un ScheduledExecutorService **con 3 hilos**.
- **Tarea 1** se ejecuta una sola vez después de 2 segundos.
- **Tarea 2** se ejecuta cada 3 segundos, independientemente de cuánto tarde en terminar.
- **Tarea 3** se ejecuta cada 4 segundos, pero el retraso se cuenta desde que terminó la ejecución anterior.
- Después de 20 segundos, se programa otra tarea que apaga el scheduler.